

PyTorch-使用卷積網路進行影像分類



製作人：甯文駿

連結：

<https://github.com/ningmao47/PyTorch/tree/main/%E4%BD%BF%E7%94%A8%E5%8D%B7%E7%A9%8D%E7%B6%B2%E8%B7%AF%E9%80%B2%E8%A1%8C%E5%BD%B1%E5%83%8F%E5%88%86%E9%A1%9E>

主要內容是關於用 **PyTorch** 框架建立與訓練 **CNN (卷積神經網路) 模型** 來辨識 **MNIST 手寫數字資料集** 的 Jupyter Notebook 程式碼。

以下是該檔案程式碼邏輯的詳細解釋：

1. 匯入套件

- **模型建立**：torch、torch.nn、torch.nn.functional
- **模型訓練**：torch.optim、StepLR
- **資料處理**：torchvision.datasets、torchvision.transforms
- **訓練監控**：SummaryWriter、tqdm
- **程式參數設定**：argparse

2. 建立 CNN 模型

CNN 模型說明

- `class Net(nn.Module)`
 - 建立一個繼承 `nn.Module` 的 CNN 模型。
- `__init__()`
 - 定義模型各層結構。
 - `self.conv1 = nn.Conv2d(1, 32, 3, 1)`
 - 第一層卷積層。
 - 輸入通道：1 (灰階圖片)。
 - 輸出通道：32 (32 個特徵圖)。
 - 卷積核大小：3x3。
 - 步長 (Stride): 1。

- `self.conv2 = nn.Conv2d(32, 64, 3, 1)`
 - 第二層卷積層。
 - 將 32 個特徵圖擴增為 64 個特徵圖。
- `self.dropout1 = nn.Dropout(0.25)`
 - Dropout 層。
 - 訓練時隨機丟棄 25% 神經元，降低過擬合 (Overfitting)。
- `self.dropout2 = nn.Dropout(0.5)`
 - 第二個 Dropout 層。
 - 訓練時隨機丟棄 50% 神經元，提高模型泛化能力。
- `self.fc1 = nn.Linear(9216, 128)`
 - 第一層全連接層 (Fully Connected)。
 - 將展平後的 9216 個特徵映射到 128 個神經元。
- `self.fc2 = nn.Linear(128, 10)`
 - 輸出層。
 - 共 10 個輸出節點，對應 10 個分類 (如 MNIST 數字 0~9)。

`forward()` 前向傳播流程

- `x = self.conv1(x)`
 - 經過第一層卷積，提取低階特徵 (如邊緣、紋理)。
- `x = F.relu(x)`
 - 使用 ReLU 激活函數，增加模型非線性能力。
- `x = self.conv2(x)`
 - 經過第二層卷積，提取更高階影像特徵。

- `x = F.relu(x)`
 - 再次使用 ReLU 激活函數。
- `x = F.max_pool2d(x, 2)`
 - 最大池化 (2x2)。
 - 降低特徵圖大小，保留重要特徵並減少運算量。
- `x = self.dropout1(x)`
 - 第一個 Dropout，降低過擬合。
- `x = torch.flatten(x, 1)`
 - 將多維特徵圖展平成一維向量，方便輸入全連接層。
- `x = self.fc1(x)`
 - 第一層全連接，整合影像特徵。
- `x = F.relu(x)`
 - 使用 ReLU 激活函數。
- `x = self.dropout2(x)`
 - 第二個 Dropout，進一步降低過擬合。
- `x = self.fc2(x)`
 - 最終分類層，輸出 10 個類別的分數 (Logits)。
- `output = F.log_softmax(x, dim=1)`
 - 將輸出轉換為各類別的對數機率 (Log Probability)，常搭配 NLLLoss 使用。
- `return output`
 - 回傳模型的最終分類結果。

模型架構流程

輸入圖片 (1×28×28)

→ Conv2D (1→32)

→ ReLU

→ Conv2D (32→64)

→ ReLU

→ Max Pooling (2×2)

→ Dropout (25%)

→ Flatten

→ Fully Connected (9216→128)

→ ReLU

→ Dropout (50%)

→ Fully Connected (128→10)

→ Log Softmax (輸出 10 類別機率)

3. train & test

train() 訓練函式

- `model.train()`
 - 將模型設為訓練模式，啟用 Dropout、Batch Normalization 等訓練功能。
- 初始化變數
 - `train_loss = 0`：累計訓練損失。
 - `correct = 0`：累計預測正確的樣本數。
- `for batch_idx, (data, target) in enumerate(tqdm(train_loader))`
 - 逐批 (Batch) 讀取訓練資料。
 - `tqdm` 顯示訓練進度條。
- `data.to(device), target.to(device)`
 - 將資料與標籤移至 CPU 或 GPU 進行運算。
- `optimizer.zero_grad()`
 - 將上一批次的梯度清除，避免梯度累積影響本次更新。

- `output = model(data)`
 - 前向傳播 (Forward)，取得模型預測結果。
- `loss = F.nll_loss(output, target)`
 - 計算損失函數 (Negative Log Likelihood Loss)。
- `loss.backward()`
 - 執行反向傳播 (Backpropagation)，計算各參數梯度。
- `optimizer.step()`
 - 根據梯度更新模型權重。
- `train_loss += loss.item() * data.size(0)`
 - 累加每個 Batch 的總損失。
- `pred = output.argmax(dim=1)`
 - 找出每筆資料預測機率最高的類別。
- `correct += pred.eq(target).sum().item()`
 - 統計預測正確的樣本數。
- 計算平均 Loss 與 Accuracy
 - `train_loss /= len(train_loader.dataset)`：平均訓練損失。
 - `train_acc = correct / len(train_loader.dataset)`：訓練準確率。
- `return train_loss, train_acc`
 - 回傳本 Epoch 的平均 Loss 與 Accuracy。

`test()` 測試函式

- `model.eval()`
 - 將模型設為測試模式，停用 Dropout 等訓練功能。

- **初始化變數**
 - `test_loss = 0`：累計測試損失。
 - `correct = 0`：累計預測正確數。
- **with torch.no_grad():**
 - 關閉梯度計算，降低記憶體使用並提升測試速度。
- **逐批讀取測試資料**
 - 將資料與標籤移至 CPU 或 GPU。
 - 執行前向傳播取得預測結果。
- **test_loss += F.nll_loss(..., reduction='sum').item()**
 - 累加所有測試資料的總損失。
- **pred = output.argmax(dim=1)**
 - 找出每筆資料預測機率最高的類別。
- **correct += pred.eq(target).sum().item()**
 - 統計預測正確的樣本數。
- **計算平均 Loss 與 Accuracy**
 - `test_loss /= len(test_loader.dataset)`：平均測試損失。
 - `test_acc = correct / len(test_loader.dataset)`：測試準確率。
- **輸出測試結果**
 - 顯示平均 Loss、正確數、總樣本數及 Accuracy。
- **return test_loss, test_acc**
 - 回傳測試 Loss 與 Accuracy。

4. main()

main() 主程式流程

- **建立 TensorBoard**
 - `SummaryWriter("runs/mnist_experiment")`
 - 用來記錄 Loss、Accuracy、模型架構等資訊，方便使用 TensorBoard 視覺化訓練過程。
- **設定訓練參數 (argparse)**
 - `batch-size`：訓練批次大小（預設 64）。
 - `test-batch-size`：測試批次大小（預設 1000）。
 - `epochs`：訓練回合數（預設 20）。
 - `lr`：學習率（Learning Rate）。
 - `gamma`：學習率衰減倍率。
 - `seed`：亂數種子，方便重現結果。
 - `save-model`：是否儲存訓練完成的模型。
- **設定運算裝置**
 - 判斷是否可使用 GPU（Accelerator）。
 - 若可使用則採用 GPU，否則使用 CPU。
- **固定亂數**
 - `torch.manual_seed(args.seed)`
 - 確保每次訓練結果具有一致性。
- **設定 DataLoader**
 - 建立訓練與測試資料載入器。
 - 若使用 GPU，額外設定：
 - `shuffle=True`：打亂訓練資料。
 - `pin_memory=True`：加快 CPU→GPU 傳輸。
 - `num_workers=1`：使用子程序載入資料。

- `persistent_workers=True`：持續保留工作程序，提高效率。
- 資料前處理 (transform)
 - `transforms.ToTensor()`：將圖片轉換為 Tensor。
 - `transforms.Normalize()`：使用 MNIST 的平均值與標準差進行標準化，加速模型收斂。
- 下載與載入 MNIST 資料集
 - `dataset1`：訓練資料集。
 - `dataset2`：測試資料集。
- 建立 DataLoader
 - `train_loader`：提供訓練資料。
 - `test_loader`：提供測試資料。
- 建立 CNN 模型
 - `model = Net().to(device)`
 - 建立自訂 CNN 並移至 CPU 或 GPU。
- 記錄模型架構
 - 使用 `writer.add_graph()` 將模型結構寫入 TensorBoard。
- 建立最佳化器 (Optimizer)
 - 使用 `optim.Adadelta()` 更新模型權重。
 - Adadelta 會根據梯度自動調整每個參數的學習率。
- 建立學習率排程器
 - StepLR
 - 每個 Epoch 將學習率乘上 `gamma`，逐步降低學習率，提升收斂效果。
- 訓練與測試迴圈

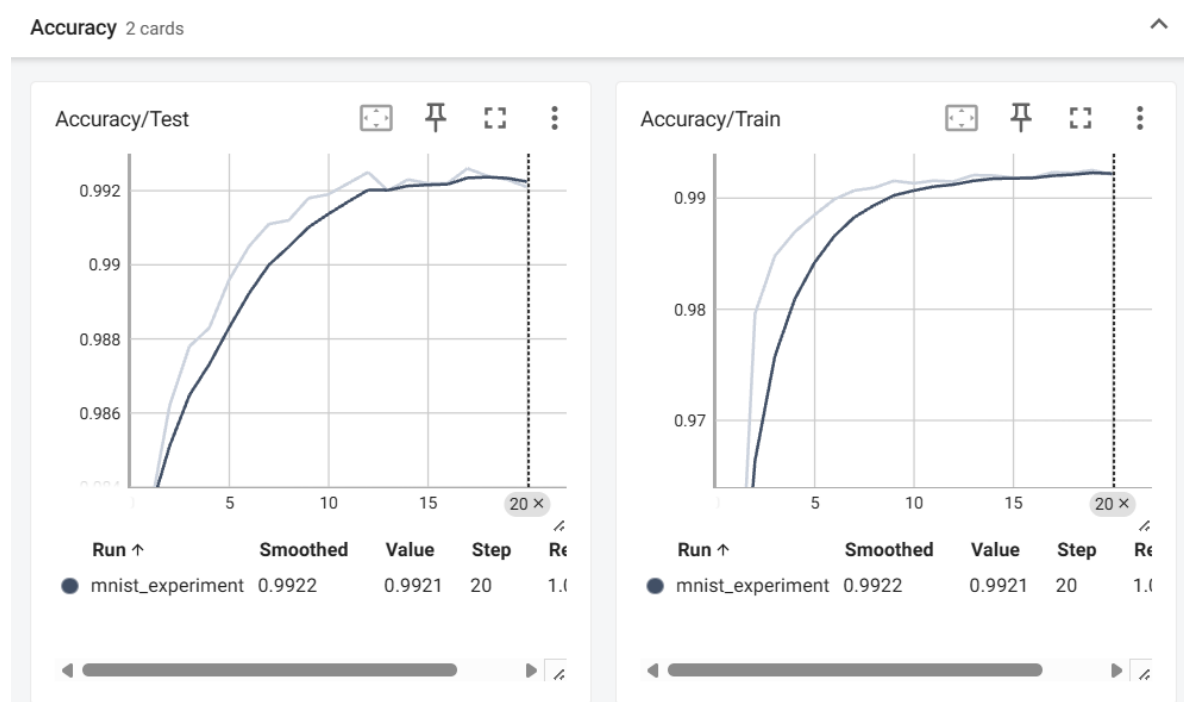
- 每個 Epoch：
 - 呼叫 `train()` 進行模型訓練。
 - 呼叫 `test()` 評估模型效能。
 - 顯示 Train/Test 的 Loss 與 Accuracy。
- 記錄 TensorBoard
 - 紀錄：
 - Train Loss
 - Test Loss
 - Train Accuracy
 - Test Accuracy
 - Learning Rate
- 更新學習率
 - `scheduler.step()`
 - 根據設定降低學習率。
- 儲存模型
 - 若設定 `--save-model`，則將模型權重儲存為 `mnist_cnn.pt`。
- 關閉 TensorBoard
 - `writer.close()`
 - 結束紀錄並釋放資源。
- 程式進入點
 - `if __name__ == '__main__':`
 - 當此程式直接執行時，呼叫 `main()` 作為整個訓練流程的入口。

5. TensorBoard

Accuracy

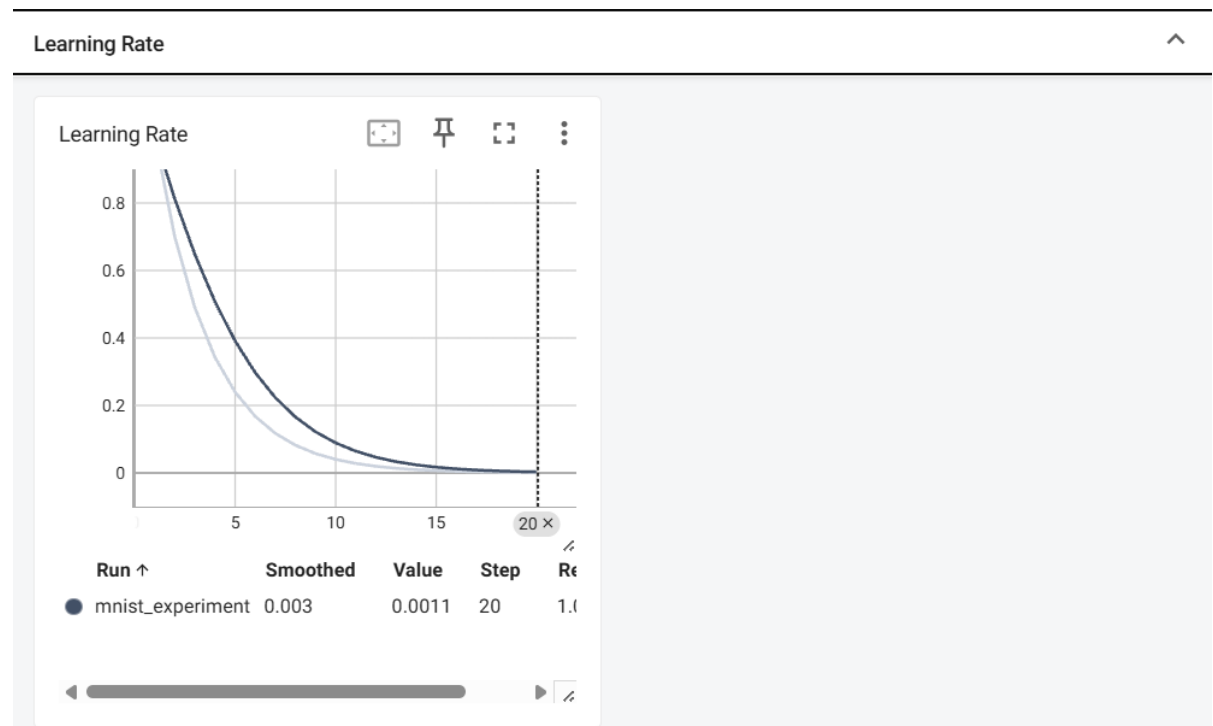
- 左圖 **Accuracy/Test**：模型在**測試集**（未見過的數據）上的準確率，用來評估模型的泛化能力。
- 右圖 **Accuracy/Train**：模型在**訓練集**（學習用的數據）上的準確率，用來評估模型學得如何。
- X 軸（Step）代表訓練步數（或 Epochs），目前顯示到第 20 步。
- Y 軸代表準確率。
- 最終結果：在第 20 步時，測試集與訓練集的平滑準確率（Smoothed）都達到了 0.9922（99.22%）。

隨著訓練步數增加，兩張圖的曲線都穩定上升並趨於平緩，且**測試集與訓練集的準確率非常接近**（皆約 99.2%）。這代表模型訓練非常成功，學得很好，而且沒有發生過擬合（Overfitting）的現象。



Learning rate

圖中的曲線隨著訓練步數（Step）增加而快速下降，這代表實驗中使用了**學習率衰減策略**。在訓練初期使用較大的學習率可以加快收斂速度，後期逐漸調小則有助於模型在最優解附近進行微調，避免震盪。



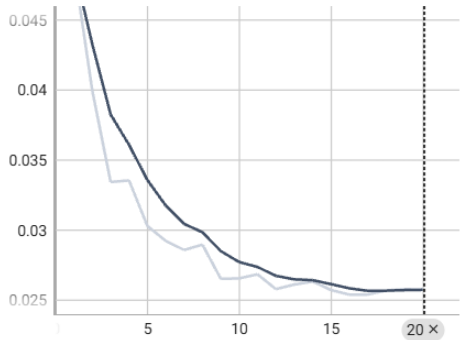
Loss

- 左圖 **Loss/Test**：模型在**測試集**上的損失值。
- 右圖 **Loss/Train**：模型在**訓練集**上的損失值。
- X 軸 (Step)：訓練步數，目前進行到第 20 步。
- Y 軸：損失值。
- 最終結果：在第 20 步時，測試集的平滑損失值 (Smoothed) 為 0.0257 (實際值 0.0258)；訓練集的平滑損失值同樣為 0.0257 (實際值 0.0262)。

兩張圖的曲線都隨著訓練步數增加而快速下降並收斂。更重要的是，**測試集損失值與訓練集損失值幾乎完全同步下降**，沒有出現「訓練集持續下降、測試集卻反彈爬升」的過擬合現象。這再次證實模型學得非常健康且穩定。



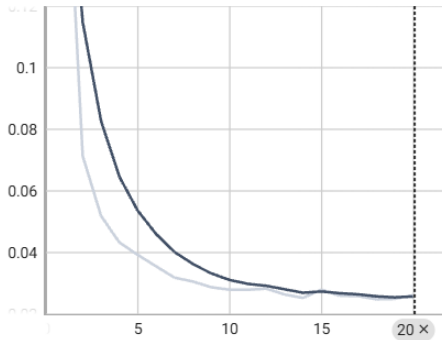
Loss/Test



Run	Smoothed	Value	Step	Relative
mnist_experiment	0.0257	0.0258	20	1.0



Loss/Train



Run	Smoothed	Value	Step	Relative
mnist_experiment	0.0257	0.0262	20	1.076 hr

